Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

Implementar nesse projeto o código do Algoritmo de Banqueiro, um sistema que evita deadlocks, com a teoria sendo apresentada em sala de aula para evitar deadlock. Nessa atividade temos que fazer o Algoritmo do Banqueiro com número de clientes e recursos variáveis e os comandos vêm dos arquivos de customer.txt e commands.txt

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
Compila e gera o executável banker	OK	rodar make no terminal
Make clean limpa os arquivos gerados	OK	make clean
Recursos lidos	OK	./banker 10 5 7
Comandos RQ, RL e * lidos do commands.txt	OK	make teste

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
make clean
make
```

Executar (exemplo):

```
./banker 10 5 7
```

4. Arquitetura (resumida)

Arquivos e responsabilidades:

1. **main.c → Coordena as etapas entre os argumentos**
2. **entrada.c / entrada.h → Lê e valida os argumentos**
3. **banker.c / banker.h → Define a struct Banco e implementa o algoritmo de segurança, os comandos RQ, RL e ***
4. **Makefile → Compila, limpa e grava o log**

Decisões técnicas:

1. **Separação feita com sem strtok → o strtok apaga os campos vazios→ o 7, ,3 e 7, ,5 são recusados**
2. **Memória dinâmica → os tamanhos não são fixos → todo caminho de erro libera o que alocou**
3. **Buffer de 4096 caracteres → evita cortar uma linha sem perceber → limite de 370 recursos por linha**
4. **Usar calloc ao invés de malloc → já cria uma matriz toda de zeros no cria_matriz → facilita no allocation**

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Validação em módulos	Toda entrada vai ser validada antes de ser usada.	

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S2			
S3			

5.2 Diário de Tentativas

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Compilar com make	Compilou como esperado		10/11	Linha 5 do evidencias.log
2	S1	./banker sem argumentos	Apontou o erro sem quebrar o código		10/11	Linha 13 do evidancias.log
3	S1	./banker 10 5 7	Funciona como deveria		10/11	Linha 17 do evidencias.log
4	S1	Fazer funções para iniciar e liberar a matriz que vai guardar os valores	Rodou como deveria		11/11	Linha 86 do evidências.log
5	S1	Testar se a saída está sendo igual ao result.txt de Marcelo	O resultado bateu com o teste de Marcelo		12/11	Linha 145 do evidencias.log
6	S1	Pedi para o Claude gerar uma bateria de testes para verificar casos de erro	Os testes rodaram e aparentem ente deram certo		12/11	A partir da linha 173 do evidencias.log
7						
8						

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

6.1 Log automático (faz o trabalho pesado por você)

No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

6.2 Prints

O código em si não deu problemas, mas eu fiz uma bateria para testar os casos onde poderia dar falha como um cliente inexistente ou espaçamentos errados, linhas vazias e um erro apenas na última linha

```
TESTE: cliente inexistente (5)
$ ./banker 10 5 7
Error: commands.txt line 1: customer 5 does not exist (valid customers: 0 to 4).
>> OK (codigo de saida 1)
------------------------------------------------------------
TESTE: asterisco com espaco
$ ./banker 10 5 7
Error: commands.txt line 1: unknown command "* " (expected RQ, RL or *).
>> OK (codigo de saida 1)
------------------------------------------------------------
TESTE: linha vazia no meio
$ ./banker 10 5 7
Error: commands.txt line 2: empty line (empty lines are only allowed at the end of the file).
>> OK (codigo de saida 1)
------------------------------------------------------------
TESTE: erro so na ultima linha (nada pode ser executado)
$ ./banker 10 5 7
Error: commands.txt line 3: customer 9 does not exist (valid customers: 0 to 4).
>> OK (codigo de saida 1)
------------------------------------------------------------
```

Versão finalizada, onde mostra os casos de sucesso e os testes passados

result.txt

```
Allocate to customer 0 the resources 0 2 1
Allocate to customer 2 the resources 3 0 2
Allocate to customer 1 the resources 2 0 0
Allocate to customer 4 the resources 0 0 2
Release from customer 0 the resources 0 1 1
Allocate to customer 3 the resources 2 1 1
MAXIMUM   |   ALLOCATION  |   NEED
7 5 3     |   0 1 0       |   7 4 3
3 2 2     |   2 0 0       |   1 2 2
9 0 2     |   3 0 2       |   6 0 0
2 2 2     |   2 1 1       |   0 1 1
4 3 3     |   0 0 2       |   4 3 1
AVAILABLE 3 3 2
The resources 3 3 2 are not enough to customer 4 request 4 0 0
Allocate to customer 1 the resources 1 0 2
The customer 0 request 0 2 0 was denied because result in an unsafe state
The customer 2 request 0 1 0 was denied because exceed its maximum allocation

```

7. Uso de IA

Onde usei IA (2–3 linhas):

- Pedi para o Claude criar um plano de trabalho, para eu me organizar e fazer essa atividade sem ficar muito apertado no tempo
- Gerar uma bateria de testes para confirmar que meu código estava funcionando e aguenta testes diversificados.
- Me ajudar a declarar o que eu deveria escrever no diário de tentativas

Prompts principais (liste, sem história):

- Salve Claude, tudo certo? Vê só, eu estou precisando de ajuda para fazer uma atividade da minha cadeira do terceiro período, Infraestrutura de Software. Meu professor passou uma atividade sobre deadlock, Eu vou te mandar aqui o arquivo que ele passou explicando a atividade. Mas deixa eu te explicar o que eu quero que você faça. Eu quero que você me explique detalhadamente o que precisa

ser feito nessa implementação, me guie em como fazer essa atividade, de forma que ela esteja a prova de falhas e de testes que queiram quebrar o código.
- Claude, você pode me ajudar criando um arquivo de script para eu rodar e testar?
- Chat, até agora, poderia me mostrar o que eu fiz de uma forma que caberia no diário de tentativas?

O que validei manualmente e como:

- Eu validei os arquivos rodando toda vez que fazia uma mudança nas funções, para garantir que estava indo de acordo com o esperado

8. Reflexão Final

Acredito que eu deveria ter focado mais na parte de entender totalmente como funciona o estado inseguro, por mais que ele não seja uma garantia que vá acontecer um deadlock, nessa implementação, caso o código identificasse que não está em estado seguro,o pedido é negado e o banco volta para o estado anterior e continua para o próximo comando. Exemplo disso é o arquivo passado por Marcelo, sobre o cliente 0 que pede 0 2 0, nesse caso o work fica 2 1 0, o que impede os clientes de terminarem. Nessa situação, o estado seria inseguro. Entendendo isso, até agora eu entendi que o estado seguro é quando existe uma ordem em que os clientes terminam, dessa forma, o deadlock fica impossível, enquanto o inseguro não tem essa garantia. Foi importante validar o commands.txt porque tinha que verificar todos os parâmetros antes de aceitar, caso desse algum erro, nenhum comando seria executado e o result.txt não ficava pela metade.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	OK
Rodei os testes e evidencias.log foi gerado	OK
Tenho prints obrigatórios	OK
Testei pelo menos um caso limite e um caso inválido	OK
Preenchi a seção de uso de IA	OK
Revisei o relatório e removi frases genéricas / vazias	OK

10. Se eu tivesse mais 2 horas

Se eu tive mais umas duas horas, eu provavelmente teria feito mais testes para garantir que estava funcionando da forma que deveria, teria ampliado o código para permitir que aceitasse espaços vazios no final da linha dos arquivos de commands e customers.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.